

Studieretningscase 2025-2026

Elev: Marius Harry Nielsen

Fag	Vejleder
Fag 1: Matematik	Navn: Tue Drøhse Mikkelsen (tdm)
Fag 2: Programmering	Navn: Jakob Scheunemann Hastrup (jsha)

Problemformulering

Hvordan kan man anvende komplekse tal og Python til at visualisere Mandelbrot, og hvordan påvirker algoritmen programmets ydeevne?

- Redegør for definitionen af komplekse tal samt de regneregler, der er nødvendige for at forstå formlen $Z_n = Z_{n-1}^2 + c$. Forklar begrebet absolutværdi for komplekse tal.
- Undersøg, hvordan Escape Time algoritmen er implementeret i Python, herunder hvordan man kan anvende Numba til parallelisering af beregningerne.
- Diskuter algoritmens tidsmæssige kompleksitet ved hjælp af Big-O, hvilken betydning skærmopløsning (width x height) og antallet af iterationer har for beregningstiden.

Aflevering: SRC'en skal afleveres senest mandag **d. 11. maj 2026 kl. 16.00** på lectio.dk.

Denne side skal inkluderes i afleveringen før forsiden.

Omfang: Opgavebesvarelsen har et omfang på 5-8 normalsider a 2400 anslag. Heri medregnes der ikke: forside, indholdsfortegnelse, noter, litteraturliste, figurer, tabeller, bilag og lignende. Bemærk at i fag med "teknisk" sprog gælder 1 sides formler/bevis, programkode eller lignende for 1 normalside og skal ikke leve op til kravet om de 2400 anslag.

Vejledning: Du har mulighed for vejledning under opgaveskrivningen, og du må gerne henvende dig til dine vejledere på skolen, eller via Lectio og mail. Vejlederne må besvare konkrete, opgavetekniske spørgsmål, men kun efter henvendelse fra dig.

Mandelbrot sættet

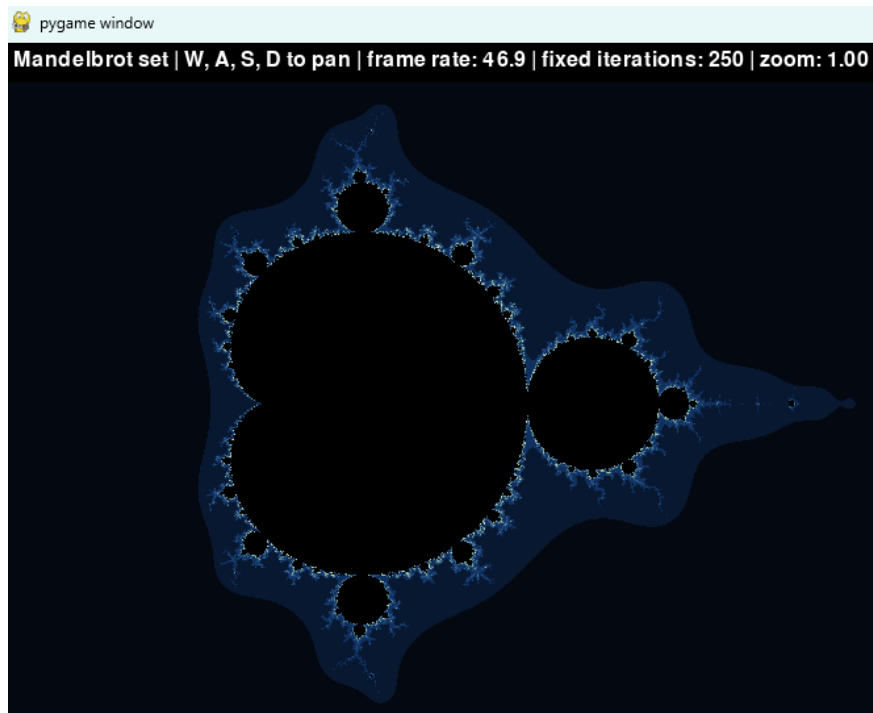
Visualisering af fraktaler med komplekse tal og Python

Marius Harry Nielsen

Studieretningscase 2025-2026

Matematik og Programmering - fraktaler

Afleveret 11. maj 2026



Indhold

Indledning.....	3
Redegørelse.....	4
Undersøgelse.....	6
Diskussion.....	8
Litteraturliste	11

Indledning

Denne studieretningscase (SRC) omhandler Mandelbrot sættet. Emnet er valgt, fordi det illustrerer, hvordan en simpel matematisk algoritme kan danne et uendeligt antal af komplekse og visuelt spændende figurer og mønstre.

Mandelbrot sættet er en visualisering af et matematisk fænomen, der opstår gennem gentagne beregninger (iterationer) med komplekse tal. Ved at benytte formlen $Z_n = Z_{n-1}^2 + c$, hvor $Z_0 = 0$, kan man bestemme, om et komplekst tal (c) er en del af Mandelbrot. Dette afgøres ved at undersøge, om talfølgen forbliver begrænset (tegnet med sort), eller om den bliver ved i uendelighed.

I dette projekt omsætter jeg den matematiske teori til et visuelt Mandelbrot sæt i Python og ser samtidig på, hvordan beregningen kan optimeres. Jeg undersøger, hvordan Escape Time-algoritmen implementeres, og hvordan man kan anvende Numba for at forbedre programmets ydeevne. Til sidst analyserer jeg algoritmens tidsmæssige kompleksitet med Big-O og ser på, hvordan skærmopløsning og antal iterationer påvirker beregningstiden.

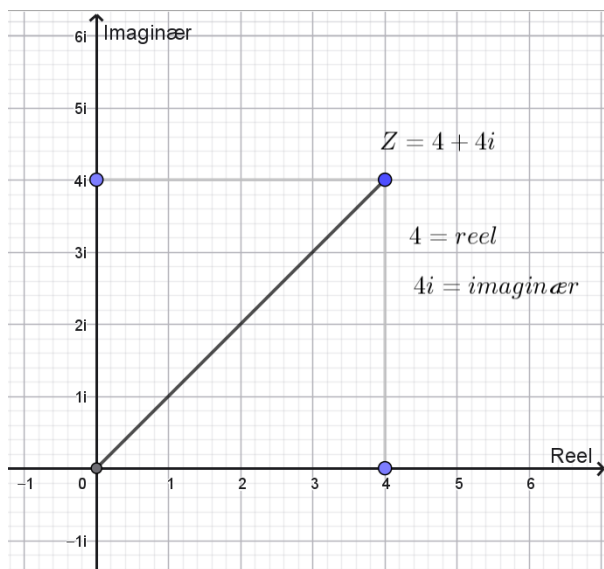
Problemformulering

Hvordan kan man anvende komplekse tal og Python til at visualisere Mandelbrot, og hvordan påvirker algoritmen programmets ydeevne?

- Redegør for definitionen af komplekse tal samt de regneregler, der er nødvendige for at forstå formlen $Z_n = Z_{n-1}^2 + c$. Forklar begrebet absolutværdi for komplekse tal.
- Undersøg, hvordan Escape Time-algoritmen er implementeret i Python, herunder hvordan man kan anvende Numba til parallelisering af beregningerne.
- Diskuter algoritmens tidsmæssige kompleksitet ved hjælp af Big-O, hvilken betydning skærmopløsning (width x height) og antallet af iterationer har for beregningstiden.

Redegørelse

Komplekse tal består af 2 elementer, en imaginær del og en reel del, $Z = a + bi$. Disse to dele kan visualiseres i et koordinatsystem, hvor den imaginære del er på y-aksen og den reelle del på x-aksen.



Inden for komplekse tal gælder $i = \sqrt{-1}$.

Komplekse tal blev indført for at kunne løse ligninger, som ikke har en løsning blandt de reelle tal. Et eksempel er ligningen $x^2 + 1 = 0$, der kræver kvadratroden af et negativt tal. Ved at indføre den imaginære enhed i , hvor $i^2 = -1$, kan man løse disse ligninger. I det udvidede talsystem har alle polynomier af grad n præcis n rødder.

Komplekse tal kan også bruges til at repræsentere punkter i et todimensionelt koordinatsystem. Det betyder, at regneoperationer med komplekse tal svarer til geometriske operationer som forskydning, drejning og skalering. Den egenskab bruges blandt andet i fysik, og i dette projekt til at beskrive Mandelbrot sættet, hvor hvert punkt i planen tolkes som et komplekst tal, der sendes igennem $Z_n = Z_{n-1}^2 + c$ formen¹.

Addition og multiplikation af komplekse tal

Komplekse tal har særlige regneregler. I mit Python program bruger jeg addition og multiplikation, så dem vil jeg kort gennemgå

- Addition²

Når man adderer komplekse tal, skal de reelle og imaginære dele adderes hver for sig.

$$Z_1 + Z_2 = (a + bi) + (c + di) = a + c + (b + d)i$$

¹ Kilde 3

² Kilde 1, afsnit Addition

- Multiplikation³

For at multiplicere komplekse tal, skal alle led ganges med hinanden.

$$Z_1 \cdot Z_2 = (a + bi) + (c + di)$$

$$= ac + adi + bci + bdi^2$$

$$= ac - bc + (bc + ad)i$$

Absolutværdi

Absolutværdien af et komplekst tal er en vigtig del af Mandelbrot sættet. Da komplekse tal kan visualiseres som punkter i et koordinatsystem, defineres absolutværdien som den direkte afstand fra origo (punktet 0,0) til tallets placering i koordinatsystemet.

For et komplekst tal, $Z = a + bi$, hvor a er den reelle del og b er den imaginære del, kan denne afstand beregnes ved hjælp af Pythagoras:

$$|Z| = \sqrt{a^2 + b^2}$$

I forbindelse med Mandelbrot algoritmen bruges absolutværdien som et stop-kriterie. Hvis absolutværdien af Z_n på et tidspunkt overstiger 2 kan det bevises, at talfølgen vil vokse mod uendelig og derfor ikke er en del af mængden.

I Python-koden tjekker programmet i stedet, om kvadratet på absolutværdien er mindre end eller lig med 4: $Z_{reel} \cdot Z_{reel} + Z_{imaginær} \cdot Z_{imaginær} \leq 4$. Det svarer til $|Z| \leq 2$, men sparer en beregning med kvadratrodd i hvert loop, hvilket er en tung operation at køre i Python. Hvis denne betingelse ikke længere er opfyldt, er punktet undsluppet, og er dermed ikke en del af sættet⁴.

³ Kilde 1, afsnit Multiplikation

⁴ Kilde 2, side 21 og 18

Undersøgelse

Escape Time i Python

```
31 coordinate_scale = 4.0 / zoom_level
32 iteration_counts = np.zeros((image_height, image_width), dtype=np.int32)
33 for y_position in prange(image_height):
34     imaginary_coordinate = center_y + (y_position - image_height / 2) * coordinate_scale / image_width
35     for x_position in range(image_width):
36         real_coordinate = center_x + (x_position - image_width / 2) * coordinate_scale / image_width
37         complex_point = complex(real_coordinate, imaginary_coordinate)
38         current_value = 0.0j
39         iteration_count = 0
40         while (current_value.real * current_value.real + current_value.imag * current_value.imag <= 4.0) and (iteration_count < max_iterations):
41             current_value = current_value * current_value + complex_point
42             iteration_count += 1
43         iteration_counts[y_position, x_position] = iteration_count
44     return iteration_counts
```

Escape Time-formlen kan blive implementeret i Python på flere forskellige måder. Jeg har forsøgt at finde den, som kræver mindst computerkraft. Jeg har fundet en version på reddit, som jeg har hentet inspiration i. Dette stykke kode gør brug af både NumPy og Numba til at effektivisere processen. Matematikken i mit program og programmet fra Reddit er principielt den samme, dog har jeg lavet en del forbedringer.

Her beskrives logikken bag `calculate_mandelbrot_iterations` funktionen:

Først beregnes skalaen for det komplekse plan baseret på `zoom_level` variabelen. Dette gøres ved at sige $coordinate_scale = \frac{4}{zoom_level}$. Udtrykket bruges til at omregne pixels til koordinater i det komplekse plan. På den måde vil et zoom level på 1 svare til at udsnittet er $[-2,2]$, som er standarden for Mandelbrot. Derefter sørger `parallel=True` og `prange` for at hver række bliver kørt på en separat CPU-kerne og derfor kører programmet hurtigere, end hvis det hele blev kørt på en enkelt kerne. Nu kan man finde det imaginære koordinat, `center_y` er centrum for udsnittet af den imaginære akse, hvor $(y_position - \frac{image_height}{2})$ flytter origo (0,0 i koordinatsystemet) til midten af skærmen. Derefter ganges resultatet med `coordinate_scale`, der sørger for at afstandene er korrekte i forhold til `zoom_level` variabelen. Herefter divideres udtrykket med `image_width` for at holde samme skala som i x-retningen, så akserne ikke er manipulerede. Det samme gøres for at finde `real_coordinate`, bare med værdier fra x-aksen i stedet for y-aksen. Så oprettes punktet i det komplekse plan ved hjælp af `complex` klassen fra NumPy.

Denne proces gentages i et while loop indtil punktet viser sig at være ustabilt eller stabilt.

I denne funktion er det tydeligt at se kodens afhængighed af NumPy til at lave det tunge matematiske arbejde. Klassen `complex` gør det muligt at springe mange trin over, som ellers skulle udføres med normal Python.

Numba og NumPy

For at kunne visualisere Mandelbrot i realtid bruger programmet to biblioteker: NumPy til datastrukturer og Numba til hastighedsoptimering.

- NumPy (Numerical Python)

NumPy er et bibliotek til matematiske beregninger i Python og er mere effektivt end normal Python til at arbejde med store, flerdimensionelle arrays.

I koden anvendes NumPy til følgende opgaver:

Databehandling af pixels: Programmet bruger `np.zeros` til at oprette et todimensionelt array, der gemmer antallet af iterationer for hver eneste pixel på skærmen.

```
32 iteration_counts = np.zeros((image_height, image_width), dtype=np.int32)
33 for y_position in prange(image_height):
34     imaginary_coordinate = center_y + (y_position - image_height / 2) * coordinate_scale / image_width
35     for x_position in range(image_width):
36         real_coordinate = center_x + (x_position - image_width / 2) * coordinate_scale / image_width
37         complex_point = complex(real_coordinate, imaginary_coordinate)
38         current_value = 0.0j
```

Vektor matematik: I filen `color_palettes.py` bruges NumPy til at udføre matematiske operationer på hele arrays samtidigt, som for eksempel normalisering af værdier og begrænsning af farvевærdier med `np.clip`.

Farver: Farvepaletten er defineret som et NumPy-array (`color_stops`), hvilket gør det hurtigt at udregne farveovergange for alle pixels på én gang.

Visualisering: Før billedet vises i Pygame, bruges `np.rot90` til at rotere arrayet, så det passer til skærmens koordinatsystem.

- Numba

Numba er en Just-In-Time (JIT) compiler, der oversætter dele af Python-koden direkte til maskinkode. Python er langsomt til store matematiske beregninger, så uden Numba ville programmet ikke kunne køre i realtid.

I koden anvendes Numba til følgende ting:

@njit dekoratøren: Funktionen `calculate_mandelbrot_iterations` er markeret med `@njit`, hvilket instruerer computeren i at compilere funktionen, før den køres.

```
22 @njit(parallel=True)
```

Parallelisering: Ved at sætte `parallel=True` og benytte `prange` (parallel range), kan programmet fordele beregningerne over alle computerens CPU-kerner samtidigt.

```
33 for y_position in prange(image_height):
```

Håndtering af komplekse tal: Numba er optimeret til at håndtere matematiske typer som `complex` og `floats`, hvilket gør den specifikke beregning af $Z_n = Z_{n+1}^2 + c$ ekstremt hurtig sammenlignet med Python.

Med NumPy til arrays og Numba til de tunge beregninger kan programmet opdatere Mandelbrot-sættet i realtid med en stabil FPS (afhængig af computerkraft).

Diskussion

Big-O notation

Big-O notation anvendes for at vurdere programmets ydeevne og skalerbarhed. Big-O beskriver algoritmens "worst-case" køretid som en funktion af inputstørrelsen. Med Mandelbrot-sættet er køretiden afhængig af tre primære variabler: skærmbredden, skærmhøjden og det maksimale antal iterationer.

Ved at tage et kikk på funktionen `calculate_mandelbrot_iterations`, kan kompleksiteten opdeles i tre niveauer:

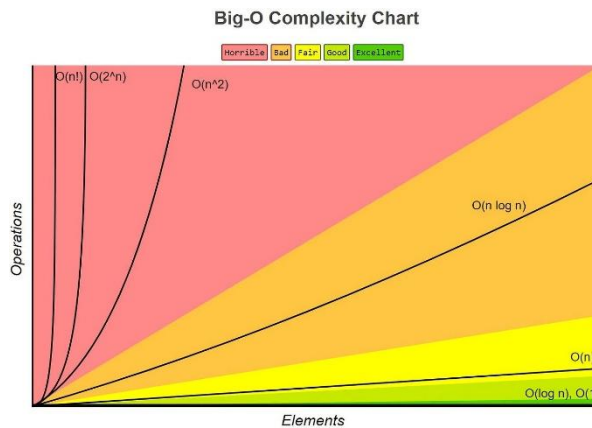
- Ydre loop: Programmet gennemgår hver eneste pixel på skærmen ved hjælp af to nestede loops (2d array). Antallet af operationer her er proportionalt med $W \cdot H$. Hvis opløsningen f.eks. er 800x600, skal algoritmen behandle 480.000 individuelle punkter.
- Indre loop: For hver pixel køres et while loop, der beregner $Z_{n+1} = z_n^2 + c$. I worst case kører dette loop I gange (bestemt af `fixed_max_iterations`).
- Konstante operationer: I den indre løkke udføres beregninger med komplekse tal og sammenligninger. Da de tager den samme mængde tid uanset input, har de en Big-O på $O(1)$.

Samlet set kan den tidsmæssige kompleksitet udtrykkes som:

$$O(W \cdot H \cdot I)$$

Det er worst case. Punkter inde i mængden kører hele I igennem, men punkter langt udenfor escaper ofte efter under 10 iterationer, så det reelle antal operationer er væsentligt lavere. Det forklarer også, hvorfor FPS ikke halveres præcist, når jeg fordobler I i Forsøg 3.

Det betyder, at køretiden vokser lineært med hver af de tre variabler hver for sig. Antallet af pixels er $W \cdot H$, så hvis man fordobler både bredden og højden samtidigt, firedobles antallet af pixels. Hvilket er det, der nogle gange beskrives som kvadratisk vækst, men det forudsætter at W og H stiger sammen.



Dette viser en vigtig begrænsning ift. hardware. Hvis man ønsker høj opløsning og samtidig høj matematisk præcision (mange iterationer), stiger den beregningsmæssige belastning.

$O(W \cdot H \cdot I)$ kan, hvis man antager at W , H og I alle skalerer med samme størrelsesorden n , skrives som $O(n^3)$, altså en vækst i 3. potens. Dette er ikke en ideel tidsmæssig kompleksitet, og skyldes at programmet gennemgår Escape Time algoritmen op til I gange pr. pixel. Og da antallet af pixels stiger med W og H , stiger gentagelserne eksplosivt. I praksis sættes I (max_iteration) uafhængigt af opløsningen, så med fast I er kompleksiteten reelt $O(W \cdot H)$, altså lineær i antallet af pixels (n^2).

Numba og kompleksitet

Jeg har i mit program gjort brug af Numba og dens `@jit(parallel=True)` funktion. Dog ændrer den ikke på selve grafen repræsenteret i Big-O. Algoritmen udfører stadig det samme antal operationer. Numba oversætter koden til maskinkode og fordeler arbejdet på flere CPU-kerner, så programmet kører hurtigere i sekunder og minutter, men selve antallet af operationer er det samme. Effekten er størst på en processor med mange kerner. I teorien ville en computer med en processor, der har 16 kerner, køre programmet dobbelt så hurtigt, som en processor med 8 kerner. Derudover er det også bundet op på den individuelle processor, og hvor god den er, men det er markant sværere at beskrive/forklare. Numba fungerer på denne måde som et workaround på den relativt dårligt optimerede Escape Time algoritmes implementering i Python.

Stress-test af mit program

Her er resultaterne efter en stress-test af mit program. Jeg har kørt det på min skole laptop, der har en 12-kerne Intel Ultra 7 CPU. FPS er målt ved max zoom uden at panorere, så testen er så stabil som muligt.

Bemærk også, at der er en FPS cap ved 60. Denne cap vil dog realistisk set ikke overskrides, medmindre man kører programmet på en urealistisk kraftig CPU, som en AMD Threadripper med 96 kerner.

	Width (W)	Height (H)	Iterationer (I)	FPS
Kontrollforsøg	800 pixels	600 pixels	250	$\approx 7,6$
Forsøg 1	1600 pixels	600 pixels	250	$\approx 3,5$
Forsøg 2	800 pixels	800 pixels	250	$\approx 5,7$
Forsøg 3	800 pixels	600 pixels	500	$\approx 5,6$
Forsøg 4	1600 pixels	800 pixels	250	$\approx 3,1$
Forsøg 5	1600 pixels	600 pixels	500	$\approx 2,7$
Forsøg 6	800 pixels	800 pixels	500	$\approx 4,7$
Forsøg 7	1600 pixels	800 pixels	500	$\approx 2,3$

Til mine forsøg har jeg testet forskellige kombinationer. En ændring har jeg valgt skal betyde en fordobling - igen for at holde et kontrolleret testmiljø. Dog kunne jeg ikke fordoble højden, på grund af skærmen på min computer, og den kunne maks have en højde på 800 pixels (hvor jeg stadig kunne se FPS counter). Efter forsøgene kan man se, at der altså er en forskel på 5,3 FPS mellem programmet, som jeg har skrevet det, og en fordobling af alle 3 parametre.

Konklusion

Formålet med denne SRC var at undersøge, hvordan komplekse tal og Python kan bruges til at visualisere Mandelbrot sættet, og hvordan algoritmen påvirker programmets ydeevne.

I redegørelsen viste jeg, at et komplekst tal $Z = a + bi$ kan tolkes som et punkt i et 2d koordinatsystem, og at addition og multiplikation af komplekse tal følger faste regneregler. Absolutværdien $|Z| = \sqrt{a^2 + b^2}$ bruges som stop-kriterie i Escape Time algoritmen: når $|Z|$ overstiger 2, vokser talfølgen mod uendelig, og punktet hører ikke til mængden. I selve koden sammenligner jeg i stedet $a^2 + b^2$ med 4, fordi det sparer en kvadratrodspr. iteration.

I undersøgelsen gennemgik jeg, hvordan Escape Time formelen $Z_n = Z_{n+1}^2 + c$ er implementeret i funktionen `calculate_mandelbrot_iterations`. Pixelkoordinater omregnes til komplekse tal via `coordinate_scale = 4 / zoom_level`, og hvert punkt itereres indtil det enten escaper eller når `max_iteration`. NumPy står for arrays og farveberegninger, mens Numba via `@jit(parallel=True)` og `prange` oversætter funktionen til maskinkode og fordeler rækkerne på flere CPU kerner.

I diskussionen analyserede jeg algoritmens tidsmæssige kompleksitet til $O(W \cdot H \cdot I)$. Hvis W, H og I skalerer sammen, svarer det til $O(n^3)$, men i praksis holdes I konstant, så den reelle kompleksitet er $O(W \cdot H)$. Stress-testen bekræftede mønsteret: FPS faldt

fra ca. 7,6 i kontrolforsøget til ca. 2,3, da alle tre parametre blev fordoblet. Det er en forskel på 5,3 FPS. Numba ændrer ikke Big-O, men sænker konstantleddet ved at parallelisere arbejdet på tværs af kernerne. Det er årsagen til, at programmet overhovedet kan køre interaktivt på en almindelig laptop.

Mandelbrot sættet viser, at et simpelt matematisk udtryk kan kræve meget regnekraft, når det skal vises i realtid. Selve algoritmen er ikke specielt effektiv, men kombinationen af NumPy og Numba gør den brugbar i praksis.

Litteraturliste

Kilde 1:

<https://matbhtx.systime.dk/?id=1656>

Kilde 2:

https://01005.compute.dtu.dk/enotes/01_-_Komplekse_Tal.pdf

Kilde 3:

<https://www.youtube.com/watch?v=ZV82k7Bu-BM>